

Week 9 demo — fallback captures

Apply the Visitor pattern: applied or just labelled?

Captured with Claude Code, Sonnet 5 (low effort), September 2026

When to use this deck

Switch here if the live AI fails (no answer after 20 seconds, network down, garbage output). Run the same verification checks and defect catalogue against these captures: the Visitor from prompt #1, the added Adapter / Decorator / Proxy from prompt #2, and the “don’t touch the vehicle classes” reserve.

Prompt #1

“Apply the Visitor pattern to compute a maintenance report across our vehicle types (Bike, E Bike, Scooter) in the bike-sharing app.”

Al's answer — the visitor interface

```
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
```

```
# ---- Visitor interface -----
```

```
class VehicleVisitor(ABC):
    @abstractmethod
    def visit_bike(self, bike: Bike): ...
    @abstractmethod
    def visit_ebike(self, ebike: EBike): ...
    @abstractmethod
    def visit_scooter(self, scooter: Scooter): ...
```

```
# ---- Elements -----
```

```
class Vehicle(ABC):
    id: str
    km_since_service: float

    @abstractmethod
    def accept(self, visitor: VehicleVisitor): ...
```

Al's answer — the vehicles

```
@dataclass
class Bike(Vehicle):
    id: str
    km_since_service: float = 0
    chain_wear_pct: float = 0

    def accept(self, visitor): return visitor.visit_bike(self)

@dataclass
class EBike(Vehicle):
    id: str
    km_since_service: float = 0
    chain_wear_pct: float = 0
    battery_health_pct: float = 100

    def accept(self, visitor): return visitor.visit_ebike(self)

@dataclass
class Scooter(Vehicle):
    id: str
    km_since_service: float = 0
    battery_health_pct: float = 100
    tire_wear_pct: float = 0

    def accept(self, visitor): return visitor.visit_scooter(self)
```

Al's answer — the report visitor

```
# --- Concrete visitor: maintenance report -----  
@dataclass  
class MaintenanceItem:  
    vehicle_id: str  
    kind: str  
    tasks: list[str]  
  
@dataclass  
class MaintenanceReportVisitor(VehicleVisitor):  
    items: list[MaintenanceItem] = field(default_factory=list)  
  
    def visit_bike(self, b: Bike):  
        tasks = []  
        if b.km_since_service > 500: tasks.append("General service")  
        if b.chain_wear_pct > 75: tasks.append("Replace chain")  
        self._add(b, "Bike", tasks)  
  
    def visit_ebike(self, e: EBike):  
        tasks = []  
        if e.km_since_service > 400: tasks.append("General service")  
        if e.chain_wear_pct > 75: tasks.append("Replace chain")  
        if e.battery_health_pct < 70: tasks.append("Battery inspection/replacement")  
        self._add(e, "EBike", tasks)
```

Al's answer — the report visitor, continued

```
def visit_scooter(self, s: Scooter):
    tasks = []
    if s.km_since_service > 300: tasks.append("General service")
    if s.tire_wear_pct > 60: tasks.append("Replace tires")
    if s.battery_health_pct < 70: tasks.append("Battery inspection/replacement")
    self._add(s, "Scooter", tasks)

def _add(self, v: Vehicle, kind: str, tasks: list[str]):
    if tasks:
        self.items.append(MaintenanceItem(v.id, kind, tasks))

def report(self) -> str:
    if not self.items:
        return "No maintenance needed."
    return "\n".join(f"[{i.kind}] {i.vehicle_id}: {' '.join(i.tasks)}"
                    for i in self.items)

# ---- Usage -----
if __name__ == "__main__":
    fleet: list[Vehicle] = [
        Bike("B-1", 620, 80),
        EBike("E-1", 100, 20, 65),
        Scooter("S-1", 350, 90, 70),
    ]
    v = MaintenanceReportVisitor()
    for vehicle in fleet:
        vehicle.accept(v)
    print(v.report())
```

Al's answer — what it said

[...] Below is a self-contained Python sketch. Rename the fields and methods to match your real classes.

[...]

How it maps to the pattern

- ▶ **Visitor:** `VehicleVisitor` has one `visit_*` method per concrete vehicle type.
- ▶ **Elements:** each vehicle's `accept` calls back the method for its own type (double dispatch). That removes `isinstance` chains.
- ▶ **Concrete visitor:** `MaintenanceReportVisitor` holds all the maintenance rules and collects the results. Vehicle classes stay free of report logic.
- ▶ **Extending:** a new report, such as cost estimation or a safety audit, is a new visitor class and needs no changes to the vehicles. A new vehicle type means adding a `visit_*` method to every visitor. That is the pattern's usual trade-off.

The thresholds (500 km, 75% wear, 70% battery, and so on) are placeholders I made up. Tell me your language and where the vehicle classes live, and I can adapt this to the real code and write it into your project.

Prompt #2 — the planned stress test

“Now also add Adapter, Decorator, and Proxy patterns to this design.”

Prompt #2 answer — the plan

I still have no file tools, so this is again code for you to paste. [...]

Each new pattern has a separate job:

- ▶ **Adapter:** makes a vendor telemetry API fit our interface.
- ▶ **Proxy:** adds caching in front of that API.
- ▶ **Decorator:** adds logging and timing around any visitor.

Prompt #2 answer — the vendor API and our interface

```
import time
from abc import ABC, abstractmethod
from dataclasses import dataclass

# ===== ADAPTER: vendor telemetry API -> our TelemetrySource =====
@dataclass
class Reading:
    km_since_service: float
    battery_health_pct: float | None = None

class TelemetrySource(ABC):
    # the interface we want (Target)
    @abstractmethod
    def read(self, vehicle_id: str) -> Reading: ...

class VendorIoTClient:
    # third-party (Adaptee), can't change
    def fetch_device_state(self, device_key: str) -> dict:
        # imagine an HTTP call; returns miles and a 0..1 battery fraction
        return {"odometer_mi": 310.0, "batt_soh": 0.64}
```

Prompt #2 answer — the Adapter

```
class VendorTelemetryAdapter(TelemetrySource):    # Adapter
    MI_TO_KM = 1.609344

    def __init__(self, client: VendorIoTClient):
        self._client = client

    def read(self, vehicle_id: str) -> Reading:
        raw = self._client.fetch_device_state(vehicle_id)
        return Reading(
            km_since_service=raw["odometer_mi"] * self.MI_TO_KM,
            battery_health_pct=raw["batt_soh"] * 100,
        )
```

Prompt #2 answer — the Proxy

```
# ===== PROXY: caching proxy in front of any TelemetrySource =====
class CachingTelemetryProxy(TelemetrySource):    # same interface as the real subject
    def __init__(self, real: TelemetrySource, ttl_seconds: float = 60):
        self._real = real
        self._ttl = ttl_seconds
        self._cache: dict[str, tuple[float, Reading]] = {}

    def read(self, vehicle_id: str) -> Reading:
        now = time.monotonic()
        hit = self._cache.get(vehicle_id)
        if hit and now - hit[0] < self._ttl:
            return hit[1]
        reading = self._real.read(vehicle_id)    # only hit the network on miss/expiry
        self._cache[vehicle_id] = (now, reading)
        return reading

def sync_vehicle(vehicle: Vehicle, source: TelemetrySource) -> None:
    r = source.read(vehicle.id)
    vehicle.km_since_service = r.km_since_service
    if r.battery_health_pct is not None and hasattr(vehicle, "battery_health_pct"):
        vehicle.battery_health_pct = r.battery_health_pct
```

Prompt #2 answer — the Decorator

```
# ===== DECORATOR: add behavior around any VehicleVisitor =====
class VisitorDecorator(VehicleVisitor):
    def __init__(self, inner: VehicleVisitor):
        self._inner = inner

    def visit_bike(self, b):      return self._inner.visit_bike(b)
    def visit_ebike(self, e):    return self._inner.visit_ebike(e)
    def visit_scooter(self, s):  return self._inner.visit_scooter(s)

    def __getattr__(self, name):    # expose inner API, e.g. report()
        return getattr(self._inner, name)

class LoggingVisitor(VisitorDecorator):
    def _log(self, kind, v, call):
        print(f"[log] visiting {kind} {v.id}")
        return call(v)

    def visit_bike(self, b):      return self._log("Bike", b, super().visit_bike)
    def visit_ebike(self, e):    return self._log("EBike", e, super().visit_ebike)
    def visit_scooter(self, s):  return self._log("Scooter", s, super().visit_scooter)
```

Prompt #2 answer — a second Decorator and the wiring

```
class TimingVisitor(VisitorDecorator):
    def __init__(self, inner):
        super().__init__(inner)
        self.elapsed = 0.0

    def _time(self, call, v):
        t = time.perf_counter()
        try:
            return call(v)
        finally:
            self.elapsed += time.perf_counter() - t

    def visit_bike(self, b):    return self._time(super().visit_bike, b)
    def visit_ebike(self, e):  return self._time(super().visit_ebike, e)
    def visit_scooter(self, s): return self._time(super().visit_scooter, s)

# ===== Wiring =====
if __name__ == "__main__":
    telemetry = CachingTelemetryProxy(VendorTelemetryAdapter(VendorIoTClient()), ttl_seconds=30)

    fleet: list[Vehicle] = [Bike("B-1", chain_wear_pct=80), EBike("E-1"), Scooter("S-1", tire_wear_pct=70)]
    for v in fleet:
        sync_vehicle(v, telemetry)

    visitor = TimingVisitor(LoggingVisitor(MaintenanceReportVisitor()))
    for v in fleet:
        v.accept(visitor)

    print(visitor.report())
    print(f"visit time: {visitor.elapsed:.6f}s")
```

Prompt #2 answer — what it said

Why each pattern sits where it does

- ▶ **Adapter:** `VendorIoTClient` returns miles and a 0–1 fraction, and its method names don't match ours. The adapter converts that to `Reading` in km and percent. Swapping vendors means writing a new adapter, and nothing else changes.
- ▶ **Proxy:** `CachingTelemetryProxy` implements `TelemetrySource`, so callers can't tell it from the real source. It adds a TTL cache to avoid a network call per vehicle per report. The same slot could hold an access-control or retry proxy.
- ▶ **Decorator:** the decorators wrap the visitor rather than the vehicles. Vehicles dispatch through `accept`, so a vehicle-side wrapper would hide its own type from the visitor. Wrapping the visitor keeps double dispatch intact, and the decorators stack in any order.

Caveats

- ▶ The mock vendor returns fixed values, so every vehicle gets the same reading. That is why B-1 shows 310 mi as about 499 km.
- ▶ `__getattr__` in `VisitorDecorator` is what lets `visitor.report()` work through the stack. An explicit `report()` pass-through is clearer if you prefer that.
- ▶ I haven't run this. I can't execute code in this session.

Reserve prompt

“Simplify the maintenance report: we don’t want to touch the vehicle classes at all.”

Reserve — the dispatch rules

```
from dataclasses import dataclass
from functools import singledispatch

# Bike, EBike, Scooter are imported exactly as they are today.
# Fields used below (km_since_service, chain_wear_pct, ...) are assumed from earlier.

@singledispatch
def maintenance_tasks(vehicle) -> list[str]:
    raise TypeError(f"No maintenance rules for {type(vehicle).__name__}")

@maintenance_tasks.register
def _(b: Bike) -> list[str]:
    tasks = []
    if b.km_since_service > 500: tasks.append("General service")
    if b.chain_wear_pct > 75:    tasks.append("Replace chain")
    return tasks

@maintenance_tasks.register
def _(e: EBike) -> list[str]:
    tasks = []
    if e.km_since_service > 400: tasks.append("General service")
    if e.chain_wear_pct > 75:    tasks.append("Replace chain")
    if e.battery_health_pct < 70: tasks.append("Battery inspection/replacement")
    return tasks
```

Reserve — the Scooter rule and the report

```
@maintenance_tasks.register
def _(s: Scooter) -> list[str]:
    tasks = []
    if s.km_since_service > 300:    tasks.append("General service")
    if s.tire_wear_pct > 60:        tasks.append("Replace tires")
    if s.battery_health_pct < 70:   tasks.append("Battery inspection/replacement")
    return tasks

def maintenance_report(fleet) -> str:
    lines = [
        f"[{type(v).__name__}] {v.id}: {' '.join(tasks)}"
        for v in fleet
        if (tasks := maintenance_tasks(v))
    ]
    return "\n".join(lines) or "No maintenance needed."
```

Reserve — what it said

Without changes to the vehicle classes, they can't get an `accept` method, so classic Visitor is out. The same idea works with external type-based dispatch. `functools.singledispatch` picks the rule by the vehicle's class, so the report logic stays outside the vehicles.

[...]

Usage: `print(maintenance_report(fleet))`

What changed

- ▶ **Vehicles:** no `accept`, no visitor interface, no edits. If `EBike` subclasses `Bike`, it inherits the `Bike` rule unless you register its own, so register the subclass explicitly.
- ▶ **Removed:** `VehicleVisitor`, `MaintenanceReportVisitor`, and the visitor decorators. Without `accept`, nothing calls the visitor. If you still want logging or timing, wrap `maintenance_tasks` with a plain function decorator.
- ▶ **Telemetry:** the Adapter and Proxy still apply. I stopped `sync_vehicle` from writing into vehicle instances, though, since you want them left alone. Instead, have the rules take a `Reading` as a second argument, for example `maintenance_tasks(vehicle, reading)`. Call `telemetry.read(v.id)` in `maintenance_report` and pass the result in.

Trade-off

- ▶ A new report type is now a new dispatch function, not a new visitor class. That is about as easy to add.
- ▶ A missing rule for a new vehicle type shows up as a `TypeError` at run time, where a real Visitor would fail at the interface. Add a test that runs every vehicle type through `maintenance_tasks` to catch that.